Device Layer

2.0.0

Generated by Doxygen 1.9.1

1 Module Index 1
1.1 Modules . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 Module Documentation 7
4.1 Device API . . . 7
4.1.1 Detailed Description . . . 10
4.1.2 Function Documentation . . . 11
4.1.2.1 allocDmaBuffer() . . . 11
4.1.2.2 clearDeviceAttributes() . . . 11
4.1.2.3 createPcieDeviceLayer() . . . 11
4.1.2.4 createSysEmuDeviceLayer() [1/2] . . . 12
4.1.2.5 createSysEmuDeviceLayer() [2/2] . . . 12
4.1.2.6 freeDmaBuffer() . . . 13
4.1.2.7 getActiveShiresNum() . . . 13
4.1.2.8 getDeviceAttribute() . . . 13
4.1.2.9 getDeviceConfig() . . . 14
4.1.2.10 getDevicesCount() . . . 14
4.1.2.11 getDeviceStateMasterMinion() . . . 14
4.1.2.12 getDeviceStateServiceProcessor() . . . 15
4.1.2.13 getDmaAlignment() . . . 15
4.1.2.14 getDmaInfo() . . . 15
4.1.2.15 getDramBaseAddress() . . . 15
4.1.2.16 getDramSize() . . . 17
4.1.2.17 getFreeCmaMemory() . . . 17
4.1.2.18 getFrequencyMHz() . . . 17
4.1.2.19 getSubmissionQueuesCount() . . . 18
4.1.2.20 getSubmissionQueueSizeMasterMinion() . . . 18
4.1.2.21 getSubmissionQueueSizeServiceProcessor() . . . 18
4.1.2.22 getTraceBufferServiceProcessor() . . . 18
4.1.2.23 getTraceBufferSizeMasterMinion() . . . 19
4.1.2.24 hintInactivity() . . . 19
4.1.2.25 receiveResponseMasterMinion() . . . 20
4.1.2.26 receiveResponseServiceProcessor() . . . 20
4.1.2.27 reinitDeviceInstance() . . . 20
4.1.2.28 sendCommandMasterMinion() . . . 21
4.1.2.29 sendCommandServiceProcessor() . . . 21
4.1.2.30 setSqThresholdMasterMinion() . . . 22

4.1.2.31 setSqThresholdServiceProcessor() 22
4.1.2.32 updateFirmwareImage() 22
4.1.2.33 waitForEpollEventsMasterMinion() 23
4.1.2.34 waitForEpollEventsServiceProcessor() 23

5 Class Documentation **25**

5.1 dev::CmdFlagMM Struct Reference 25
5.1.1 Detailed Description 25
5.2 dev::CmdFlagSP Struct Reference 25
5.2.1 Detailed Description 26
5.3 dev::DeviceConfig Struct Reference 26
5.3.1 Detailed Description 27
5.4 dev::DeviceLayerMock Class Reference 27
5.5 dev::DmaInfo Struct Reference 28
5.5.1 Detailed Description 28
5.6 dev::Exception Class Reference 29
5.7 dev::IDeviceAsync Class Reference 29
5.8 dev::IDeviceLayer Class Reference 30
5.9 dev::IDeviceSync Class Reference 31
5.9.1 Detailed Description 32

Index **33**

Chapter 1

Module Index

1.1 Modules

Here is a list of all modules:

Device API 7

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

dev::CmdFlagMM 25
dev::CmdFlagSP 25
dev::DeviceConfig 26
dev::DmaInfo 28
dev::IDeviceAsync 29
  dev::IDeviceLayer 30
    dev::DeviceLayerMock 27
dev::IDeviceSync 31
  dev::IDeviceLayer 30
dbg::StackException
  dev::Exception 29

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

dev::CmdFlagMM
This struct contains possible command flags for MM 25
dev::CmdFlagSP
This struct contains possible command flags for SP 25
dev::DeviceConfig
NOTE: New enum/struct entities should be added in the end for backward compatibility 26
dev::DeviceLayerMock 27
dev::DmaInfo
This struct contains the limitations / optimal DMA parameters 28
dev::Exception 29
dev::IDeviceAsync 29
dev::IDeviceLayer 30
dev::IDeviceSync
This should be a command Interface class which is shared between Ops and Management, although will be different 31

Chapter 4

Module Documentation

4.1 Device API

Classes

- struct dev::DeviceConfig

  NOTE: New enum/struct entities should be added in the end for backward compatibility.
- struct dev::DmaInfo

  This struct contains the limitations / optimal DMA parameters.
- struct dev::CmdFlagSP

  This struct contains possible command flags for SP.
- struct dev::CmdFlagMM

  This struct contains possible command flags for MM.
- class dev::Exception
- class dev::IDeviceAsync
- class dev::IDeviceSync

  This should be a command Interface class which is shared between Ops and Management, although will be different.
- class dev::IDeviceLayer

Enumerations

- enum class **FormFactor** { **PCIE** , **M2** }
- enum class **ArchRevision** { **ETSOC1** , **PANTERO** , **GEPARDO** }
- enum class dev::DeviceState {
  Ready , **PendingCommands** , **NotResponding** , **ResetInProgress** ,
  NotReady , **Undefined** }

  This enum contains possible device states.
- enum class dev::TraceBufferType {
  TraceBufferSP = 0 , **TraceBufferMM** , **TraceBufferCM** , **TraceBufferSPStats** ,
  TraceBufferMMStats , **TraceBufferTypeNum** }

  This enum contains possible trace buffer types to extract from SP.

Functions

- virtual bool dev::IDeviceAsync::sendCommandMasterMinion (int device, int sqIdx, std::byte *command, size_t commandSize, CmdFlagMM flags)=0

  Sends a command to the master minion. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsMasterMinion()`
- virtual void dev::IDeviceAsync::setSqThresholdMasterMinion (int device, int sqIdx, uint32_t bytesNeeded)=0

  Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeMasterMinion()`.
- virtual void dev::IDeviceAsync::waitForEpollEventsMasterMinion (int device, uint64_t &sq_bitmap, bool &cq↩_available, std::chrono::milliseconds timeout=std::chrono::seconds(10))=0

  Blocks the caller thread until submission queues or Completion Queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.
- virtual bool dev::IDeviceAsync::receiveResponseMasterMinion (int device, std::vector< std::byte > &response)=0

  Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsMasterMinion()`
- virtual bool dev::IDeviceAsync::sendCommandServiceProcessor (int device, std::byte *command, size_↩t commandSize, CmdFlagSP flags)=0

  Sends a command to the service processor. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsServiceProcessor()`
- virtual void dev::IDeviceAsync::setSqThresholdServiceProcessor (int device, uint32_t bytesNeeded)=0

  Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeServiceProcessor()`.
- virtual void dev::IDeviceAsync::waitForEpollEventsServiceProcessor (int device, bool &sq_available, bool &cq_available, std::chrono::milliseconds timeout=std::chrono::seconds(10))=0

  Blocks the caller thread until submission queue or completion queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.
- virtual bool dev::IDeviceAsync::receiveResponseServiceProcessor (int device, std::vector< std::byte > &response)=0

  Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsServiceProcessor()`
- virtual dev::IDeviceAsync::~IDeviceAsync ()=default

  Virtual Destructor to enable polymorphic release of the IDeviceAsync instances.
- virtual DmaInfo dev::IDeviceSync::getDmaInfo (int device) const =0

  Returns the DMA parameters.
- virtual int dev::IDeviceSync::getDevicesCount () const =0

  Returns the number of esperanto devices connected to the host.
- virtual int dev::IDeviceSync::getSubmissionQueuesCount (int device) const =0

  Returns the number of submission queues associated to given device.
- virtual DeviceState dev::IDeviceSync::getDeviceStateMasterMinion (int device) const =0

  Gets device state associated to given Master Minion device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::NotResponding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.
- virtual DeviceState dev::IDeviceSync::getDeviceStateServiceProcessor (int device) const =0

  Gets device state associated to given Service Processor device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::NotResponding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.
- virtual size_t dev::IDeviceSync::getSubmissionQueueSizeMasterMinion (int device) const =0

Returns the submission queue buffer size in bytes associated to given device all submission queues on Master Minion are equal in size.

- virtual size_t dev::IDeviceSync::getSubmissionQueueSizeServiceProcessor (int device) const =0

  Returns the submission queue buffer size in bytes associated to given device.
- virtual size_t dev::IDeviceSync::getTraceBufferSizeMasterMinion (int device, TraceBufferType traceType)=0

  Receives Master Minion's trace buffer sizes associated to given device.
- virtual bool dev::IDeviceSync::getTraceBufferServiceProcessor (int device, TraceBufferType traceType, std↩::vector< std::byte > &traceBuf)=0

  Receives Service Processor's trace buffer associated to given device in traceBuf.
- virtual int dev::IDeviceSync::updateFirmwareImage (int device, std::vector< unsigned char > &fwImage)=0

  Writes firmware image on DRAM of given device.
- virtual int dev::IDeviceSync::getDmaAlignment () const =0

  Returns the DMA alignment requirement.
- virtual uint64_t dev::IDeviceSync::getDramSize (int device) const =0

  Returns the DRAM available size (in bytes)
- virtual uint64_t dev::IDeviceSync::getDramBaseAddress (int device) const =0

  Returns the DRAM base address.
- virtual dev::IDeviceSync::~IDeviceSync ()=default

  Virtual Destructor to enable polymorphic release of the IDeviceOpsSync instances.
- virtual void * dev::IDeviceSync::allocDmaBuffer (int device, size_t sizeInBytes, bool writeable)=0

  Allocates a consecutive chunk of memory, able to do DMA. This chunk of memory must be deallocated properly using the freeDmaBuffer function.
- virtual void dev::IDeviceSync::freeDmaBuffer (void *dmaBuffer)=0

  Deallocates a previously allocated dmaBuffer.
- virtual DeviceConfig dev::IDeviceSync::getDeviceConfig (int device)=0

  returns a DeviceConfig struct containing device configured details
- virtual int dev::IDeviceSync::getActiveShiresNum (int device)=0

  returns a number of active shires (number of bits set to 1 in shire mask)
- virtual uint32_t dev::IDeviceSync::getFrequencyMHz (int device)=0

  returns device frequency
- virtual size_t dev::IDeviceSync::getFreeCmaMemory () const =0

  returns the available CMA memory in the system. That memory is allocated using allocDmaBuffer
- virtual std::string dev::IDeviceSync::getDeviceAttribute (int device, std::string relAttrPath) const =0

  Get device statistics by reading the attribute files.
- virtual void dev::IDeviceSync::clearDeviceAttributes (int device, std::string relGroupPath) const =0

  Clear device statistics in the attribute files.
- virtual void dev::IDeviceSync::reinitDeviceInstance (int device, bool masterMinionOnly, std::chrono↩::milliseconds timeout)

  Reinitialize the device master minion or both master minion and service processor NOTE: During master minion reinitialization other master minion APIs cannot be used. During both devices' reinitialization no other API should be used.
- virtual void dev::IDeviceSync::hintInactivity (int device)

  Hints the underlying device that we don't have anything to process, so it could potentially go idle / low power consumption.
- static std::unique_ptr< IDeviceLayer > dev::IDeviceLayer::createSysEmuDeviceLayer (const emu::Sys↩EmuOptions &options, uint8_t numDevices=1)

  Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.
- static std::unique_ptr< IDeviceLayer > dev::IDeviceLayer::createSysEmuDeviceLayer (std::vector< emu::↩SysEmuOptions > options)

  Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.
- static std::unique_ptr< IDeviceLayer > dev::IDeviceLayer::createPcieDeviceLayer (bool enableMaster↩Minion=true, bool enableServiceProcessor=false)

  Factory method to instantiate a IDeviceApi implementation, based on Pcie backend.
- virtual dev::IDeviceLayer::~IDeviceLayer ()=default

  Virtual Destructor to enable polymorphic release of the IDeviceApi instances.

Variables

- FormFactor dev::DeviceConfig::formFactor_

  device form factor
- uint8_t dev::DeviceConfig::tdp_

  TDP in Watts.
- uint32_t dev::DeviceConfig::totalL3Size_

  total size of L3 cache in KBytes
- uint32_t dev::DeviceConfig::totalL2Size_

  total size of L2 cache in KBytes
- uint32_t dev::DeviceConfig::totalScratchPadSize_

  total scratchpad size in KBytes
- uint8_t dev::DeviceConfig::cacheLineSize_

  chache line size, in Bytes
- uint8_t dev::DeviceConfig::numL2CacheBanks_

  num L2 cache banks
- uint32_t dev::DeviceConfig::ddrBandwidth_

  DDR bandwidth in MBytes/s.
- uint32_t dev::DeviceConfig::minionBootFrequency_

  minion boot frequency in Mhz
- uint32_t dev::DeviceConfig::computeMinionShireMask_

  mask which indicates what are the compute minion shires
- uint8_t dev::DeviceConfig::spareComputeMinionoShireId_

  spare compute minion Shire ID
- ArchRevision dev::DeviceConfig::archRevision_

  architecture revision
- uint64_t dev::DmaInfo::maxElementSize_

  maximum amount of memory that can be transfer per each DMA command entry
- uint64_t dev::DmaInfo::maxElementCount_

  max number of DMA entries per DMA command
- bool **dev::CmdFlagSP::isMmReset_** = false
- bool **dev::CmdFlagSP::isEtsocReset_** = false
- bool **dev::CmdFlagMM::isDma_** = false
- bool **dev::CmdFlagMM::isHpSq_** = false

4.1.1 Detailed Description

The device API provides an asynchronous API (command send and response) and a synchronous API (configuration discovery methods). The asynchronous API is meant to be used to send commands to a device in non-blocking way. The synchronous API offers different methods which will return inmediatly, these methods are queries about hardware configuration.

Thread safety: the API implementation shall support sending commands / receiving responses from the same thread to master minion and potentially a different thread for the service processor. Hence, the API implementation will guarantee thread safety only between 2 threads, each one using the master minion services and service processor services. Further clarification: if 2+ threads consume services for the master minion, this can lead to data races. Same for service processor.

Any error will be reported throwing an Exception of kind dev::Exception. Except for device errors which can be reported through the command response; hence in the asynchronous API one could expect an exception only if the transport layer fails, not the command execution at the device.

4.1.2 Function Documentation

4.1.2.1 allocDmaBuffer()

```
virtual void* dev::IDeviceSync::allocDmaBuffer (
            int device,
            size_t sizeInBytes,
            bool writeable )  [pure virtual]
```

Allocates a consecutive chunk of memory, able to do DMA. This chunk of memory must be deallocated properly using the freeDmaBuffer function.

Parameters

in	*sizeInBytes*	size, in bytes, for the memory allocation.
in	*writeable*	indicates if the memory should be writeable or, if false, readonly.

Returns

a chunk of memory which is suitable to be used in DMA operations.

4.1.2.2 clearDeviceAttributes()

```
virtual void dev::IDeviceSync::clearDeviceAttributes (
            int device,
            std::string relGroupPath ) const  [pure virtual]
```

Clear device statistics in the attribute files.

Parameters

in	*device*	the device which will be queried
in	*relGroupPath*	the relative group path of attributes The attribute file /sys/bus/pci/devices/[[[[<domain>]:]<bus>]:][<slot>][.[<func>]]/<relGroup↩ Path>/clear should exist and this call will clear all attributes in <relGroupPath> directory.

4.1.2.3 createPcieDeviceLayer()

```
static std::unique_ptr<IDeviceLayer> dev::IDeviceLayer::createPcieDeviceLayer (
            bool enableMasterMinion = true,
            bool enableServiceProcessor = false )  [static]
```

Factory method to instantiate a IDeviceApi implementation, based on Pcie backend.

Parameters

in	*enableMasterMinion*	if this is true, deviceLayer will attempt to open the operations port (Master Minion). If this is not enabled, then all Master Minion operations will fail.
in	*enableServiceProcessor*	if this is true, deviceLayer will attempt to open the management port (Service Processor). If this is not enabled, then all Service Processor operations will fail.

Returns

std::unique_ptr<IDeviceApi> is the IDeviceApi implementation

4.1.2.4 createSysEmuDeviceLayer() [1/2]

```
static std::unique_ptr<IDeviceLayer> dev::IDeviceLayer::createSysEmuDeviceLayer (
            const emu::SysEmuOptions & options,
            uint8_t numDevices = 1 )  [static]
```

Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.

Parameters

in	*options*	this contains all sysemu parametrizable options see emu::SysEmuOptions
in	*numDevices*	number of devices which one would like to emulate. Defaults to 1

Returns

std::unique_ptr<IDeviceApi> is the IDeviceApi implementation

4.1.2.5 createSysEmuDeviceLayer() [2/2]

```
static std::unique_ptr<IDeviceLayer> dev::IDeviceLayer::createSysEmuDeviceLayer (
            std::vector< emu::SysEmuOptions > options )  [static]
```

Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.

Parameters

in	*options*	this contains all sysemu parametrizable options see emu::SysEmuOptions. For each sysemuOptions it will create an emulation device. This method is intended to provide different sysemu parameters for each sysemu device simulated.

Returns

std::unique_ptr<IDeviceApi> is the IDeviceApi implementation

4.1.2.6 freeDmaBuffer()

```
virtual void dev::IDeviceSync::freeDmaBuffer (
            void * dmaBuffer )  [pure virtual]
```

Deallocates a previously allocated dmaBuffer.

Parameters

in	*dmaBuffer*	the buffer to be deallocated.

4.1.2.7 getActiveShiresNum()

```
virtual int dev::IDeviceSync::getActiveShiresNum (
            int device )  [pure virtual]
```

returns a number of active shires (number of bits set to 1 in shire mask)

Parameters

in	*device*	the device which will be queried

4.1.2.8 getDeviceAttribute()

```
virtual std::string dev::IDeviceSync::getDeviceAttribute (
            int device,
            std::string relAttrPath ) const  [pure virtual]
```

Get device statistics by reading the attribute files.

Parameters

in	*device*	the device which will be queried
in	*relAttrPath*	the attribute path relative to directory: /sys/bus/pci/devices/[[[[<domain>]:]<bus>]:][<slot>][.[<func>]]

Returns

the content of requested attribute file

4.1.2.9 getDeviceConfig()

```
virtual DeviceConfig dev::IDeviceSync::getDeviceConfig (
            int device )  [pure virtual]
```

returns a DeviceConfig struct containing device configured details

Parameters

in	*device*	the device which will be queried

4.1.2.10 getDevicesCount()

```
virtual int dev::IDeviceSync::getDevicesCount ( ) const  [pure virtual]
```

Returns the number of esperanto devices connected to the host.

Returns

the number of devices

4.1.2.11 getDeviceStateMasterMinion()

```
virtual DeviceState dev::IDeviceSync::getDeviceStateMasterMinion (
            int device ) const  [pure virtual]
```

Gets device state associated to given Master Minion device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::NotResponding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.

Returns

enum DeviceState

4.1.2.12 getDeviceStateServiceProcessor()

```
virtual DeviceState dev::IDeviceSync::getDeviceStateServiceProcessor (
            int device ) const  [pure virtual]
```

Gets device state associated to given Service Processor device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::Not↩ Responding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.

Returns

enum DeviceState

4.1.2.13 getDmaAlignment()

```
virtual int dev::IDeviceSync::getDmaAlignment ( ) const  [pure virtual]
```

Returns the DMA alignment requirement.

Returns

User DRAM alignment(in bits)

4.1.2.14 getDmaInfo()

```
virtual DmaInfo dev::IDeviceSync::getDmaInfo (
            int device ) const  [pure virtual]
```

Returns the DMA parameters.

Parameters

in	*device*	indicating which device to get the dma parameters from.

Returns

DmaInfo contains the dma parameters.

4.1.2.15 getDramBaseAddress()

```
virtual uint64_t dev::IDeviceSync::getDramBaseAddress (
            int device ) const  [pure virtual]
```

Returns the DRAM base address.

Parameters

in	*device*	the device which will be queried

Returns

DRAM Host Managed base address

4.1.2.16 getDramSize()

```
virtual uint64_t dev::IDeviceSync::getDramSize (
            int device ) const  [pure virtual]
```

Returns the DRAM available size (in bytes)

Parameters

in	*device*	the device which will be queried

Returns

DRAM size (in bytes)

4.1.2.17 getFreeCmaMemory()

```
virtual size_t dev::IDeviceSync::getFreeCmaMemory ( ) const  [pure virtual]
```

returns the available CMA memory in the system. That memory is allocated using allocDmaBuffer

Returns

the size in bytes of allocatable CMA memory

DEPRECATED: To be removed

4.1.2.18 getFrequencyMHz()

```
virtual uint32_t dev::IDeviceSync::getFrequencyMHz (
            int device )  [pure virtual]
```

returns device frequency

Parameters

in	*device*	the device which will be queried

4.1.2.19 getSubmissionQueuesCount()

```
virtual int dev::IDeviceSync::getSubmissionQueuesCount (
            int device ) const  [pure virtual]
```

Returns the number of submission queues associated to given device.

Returns

the number of virtual queues

4.1.2.20 getSubmissionQueueSizeMasterMinion()

```
virtual size_t dev::IDeviceSync::getSubmissionQueueSizeMasterMinion (
            int device ) const  [pure virtual]
```

Returns the submission queue buffer size in bytes associated to given device all submission queues on Master Minion are equal in size.

Returns

the submission queue size in bytes

4.1.2.21 getSubmissionQueueSizeServiceProcessor()

```
virtual size_t dev::IDeviceSync::getSubmissionQueueSizeServiceProcessor (
            int device ) const  [pure virtual]
```

Returns the submission queue buffer size in bytes associated to given device.

Returns

the submission queue size in bytes

4.1.2.22 getTraceBufferServiceProcessor()

```
virtual bool dev::IDeviceSync::getTraceBufferServiceProcessor (
            int device,
            TraceBufferType traceType,
            std::vector< std::byte > & traceBuf )  [pure virtual]
```

Receives Service Processor's trace buffer associated to given device in traceBuf.

Parameters

in	*device*	indicating which device to receive the buffer from.
in	*traceType*	indicating the type of trace buffer whose size is needed. Supported types are TraceBufferType::TraceBufferMM, TraceBufferType::TraceBufferCM.
out	*traceBuf*	buffer containing Service Processor's trace buffer.

Returns

false if there was no buffer to be received.

4.1.2.23 getTraceBufferSizeMasterMinion()

```
virtual size_t dev::IDeviceSync::getTraceBufferSizeMasterMinion (
            int device,
            TraceBufferType traceType )  [pure virtual]
```

Receives Master Minion's trace buffer sizes associated to given device.

Parameters

in	*device*	indicating which device to receive the response from.
in	*traceType*	indicating which trace buffer type to pull from device. Supported types are TraceBufferType::TraceBufferSP, TraceBufferType::TraceBufferMM, TraceBufferType::CM.

Returns

size of trace buffer.

4.1.2.24 hintInactivity()

```
virtual void dev::IDeviceSync::hintInactivity (
            int device )  [virtual]
```

Hints the underlying device that we don't have anything to process, so it could potentially go idle / low power consumption.

Parameters

in	*device*	the device to be hinted

4.1.2.25 receiveResponseMasterMinion()

```
virtual bool dev::IDeviceAsync::receiveResponseMasterMinion (
            int device,
            std::vector< std::byte > & response )  [pure virtual]
```

Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsMasterMinion()`

Parameters

in	*device*	indicating which device to receive the response from.
out	*response*	buffer containing the response.

Returns

false if there was no response to be received.

4.1.2.26 receiveResponseServiceProcessor()

```
virtual bool dev::IDeviceAsync::receiveResponseServiceProcessor (
            int device,
            std::vector< std::byte > & response )  [pure virtual]
```

Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsServiceProcessor()`

Parameters

in	*device*	indicating which device to receive the response from.
out	*response*	buffer containing the response.

Returns

false if there was no response to be received.

4.1.2.27 reinitDeviceInstance()

```
virtual void dev::IDeviceSync::reinitDeviceInstance (
            int device,
            bool masterMinionOnly,
            std::chrono::milliseconds timeout )  [virtual]
```

Reinitialize the device master minion or both master minion and service processor NOTE: During master minion reinitialization other master minion APIs cannot be used. During both devices' reinitialization no other API should be used.

Parameters

in	*device*	the device to be reinitialized
in	*masterMinionOnly*	if true only master minion will be reinitialized otherwise both master minion and service processor will be reinitialized.
in	*timeout*	the operation will be aborted if time exceeds given timeout

4.1.2.28 sendCommandMasterMinion()

```
virtual bool dev::IDeviceAsync::sendCommandMasterMinion (
            int device,
            int sqIdx,
            std::byte * command,
            size_t commandSize,
            CmdFlagMM flags )  [pure virtual]
```

Sends a command to the master minion. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsMasterMinion()`

Parameters

in	*device*	indicating which device to send the command.
in	*sqIdx*	indicates which submission queue to send the command to.
in	*command*	its a buffer which contains the command itself.
in	*commandSize*	the size of the command + payload buffer.
in	*flags*	indicates command options as defined by `CmdFlagMM`. Needed for PCIe deviceLayer

Returns

false if there was not enough space to send the command, true otherwise

4.1.2.29 sendCommandServiceProcessor()

```
virtual bool dev::IDeviceAsync::sendCommandServiceProcessor (
            int device,
            std::byte * command,
            size_t commandSize,
            CmdFlagSP flags )  [pure virtual]
```

Sends a command to the service processor. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsServiceProcessor()`

Parameters

in	*device*	indicating which device to send the command.
in	*command*	its a buffer which contains the command itself.
in	*commandSize*	the size of the command + payload buffer.
in	*flags*	indicates command options as defined by `CmdFlagSO`. Needed for PCIe deviceLayer implementations

Returns

false if there was not enough space to send the command, true otherwise

4.1.2.30 setSqThresholdMasterMinion()

```
virtual void dev::IDeviceAsync::setSqThresholdMasterMinion (
            int device,
            int sqIdx,
            uint32_t bytesNeeded )  [pure virtual]
```

Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeMasterMinion()`.

Parameters

in	*device*	indicating which device to send the command.
in	*sqIdx*	indicates which submission queue to set the threshold for.
in	*bytesNeeded*	the number of bytes needed free on Submission Queue. Should be positive and within the size returned by `getSubmissionQueueSizeMasterMinion()`

4.1.2.31 setSqThresholdServiceProcessor()

```
virtual void dev::IDeviceAsync::setSqThresholdServiceProcessor (
            int device,
            uint32_t bytesNeeded )  [pure virtual]
```

Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeServiceProcessor()`.

Parameters

in	*device*	indicating which device to send the command.
in	*sqIdx*	indicates which submission queue to set the threshold for.
in	*bytesNeeded*	the number of bytes needed free on Submission Queue. Should be positive and within the size returned by `getSubmissionQueueSizeServiceProcessor()`

4.1.2.32 updateFirmwareImage()

```
virtual int dev::IDeviceSync::updateFirmwareImage (
            int device,
            std::vector< unsigned char > & fwImage )  [pure virtual]
```

Writes firmware image on DRAM of given device.

Parameters

in	*device*	indicating which device's DRAM to be written.
in	*firmware*	image to be written

Returns

bytes written

4.1.2.33 waitForEpollEventsMasterMinion()

```
virtual void dev::IDeviceAsync::waitForEpollEventsMasterMinion (
            int device,
            uint64_t & sq_bitmap,
            bool & cq_available,
            std::chrono::milliseconds timeout = std::chrono::seconds(10) )  [pure virtual]
```

Blocks the caller thread until submission queues or Completion Queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.

Parameters

in	*device*	indicating which device to wait on for EPOLL events.
out	*sq_bitmap*	indicates which submission queue(s) became available since the last call.
out	*cq_available*	indicates if Completion Queue became available since the last call.
in	*timeout*	the operation will be aborted if no event happen in given timeout

4.1.2.34 waitForEpollEventsServiceProcessor()

```
virtual void dev::IDeviceAsync::waitForEpollEventsServiceProcessor (
            int device,
            bool & sq_available,
            bool & cq_available,
            std::chrono::milliseconds timeout = std::chrono::seconds(10) )  [pure virtual]
```

Blocks the caller thread until submission queue or completion queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.

Parameters

in	*device*	indicating which device to wait on for EPOLL events.
out	*sq_available*	indicates if submission queue became available since the last call.
out	*cq_available*	indicates if Completion Queue became available since the last call.
in	*timeout*	the operation will be aborted if no event happen in given timeout

Chapter 5

Class Documentation

5.1 dev::CmdFlagMM Struct Reference

This struct contains possible command flags for MM.

```
#include <IDeviceLayer.h>
```

Public Attributes

- bool **isDma_** = false
- bool **isHpSq_** = false

5.1.1 Detailed Description

This struct contains possible command flags for MM.

The documentation for this struct was generated from the following file:

- IDeviceLayer.h

5.2 dev::CmdFlagSP Struct Reference

This struct contains possible command flags for SP.

```
#include <IDeviceLayer.h>
```

Public Attributes

- bool **isMmReset_** = false
- bool **isEtsocReset_** = false

5.2.1 Detailed Description

This struct contains possible command flags for SP.

The documentation for this struct was generated from the following file:

- IDeviceLayer.h

5.3 dev::DeviceConfig Struct Reference

NOTE: New enum/struct entities should be added in the end for backward compatibility.

```
#include <IDeviceLayer.h>
```

Public Types

- enum class **FormFactor** { **PCIE** , **M2** }
- enum class **ArchRevision** { **ETSOC1** , **PANTERO** , **GEPARDO** }

Public Attributes

- FormFactor formFactor_

  device form factor
- uint8_t tdp_

  TDP in Watts.
- uint32_t totalL3Size_

  total size of L3 cache in KBytes
- uint32_t totalL2Size_

  total size of L2 cache in KBytes
- uint32_t totalScratchPadSize_

  total scratchpad size in KBytes
- uint8_t cacheLineSize_

  chache line size, in Bytes
- uint8_t numL2CacheBanks_

  num L2 cache banks
- uint32_t ddrBandwidth_

  DDR bandwidth in MBytes/s.
- uint32_t minionBootFrequency_

  minion boot frequency in Mhz
- uint32_t computeMinionShireMask_

  mask which indicates what are the compute minion shires
- uint8_t spareComputeMinionoShireId_

  spare compute minion Shire ID
- ArchRevision archRevision_

  architecture revision

5.3.1 Detailed Description

NOTE: New enum/struct entities should be added in the end for backward compatibility.

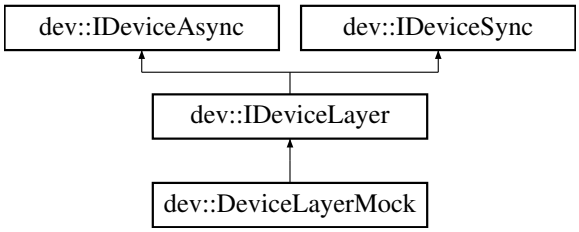
This struct contains device configured information

The documentation for this struct was generated from the following file:

- IDeviceLayer.h

5.4 dev::DeviceLayerMock Class Reference

Inheritance diagram for dev::DeviceLayerMock:



Public Member Functions

- **DeviceLayerMock** (IDeviceLayer *delegate)
- **MOCK_METHOD5** (sendCommandMasterMinion, bool(int device, int sqIdx, std::byte *command, size_t commandSize, CmdFlagMM flags))
- **MOCK_METHOD3** (setSqThresholdMasterMinion, void(int device, int sqIdx, uint32_t bytesNeeded))
- **MOCK_METHOD3** (waitForEpollEventsMasterMinion, void(int device, uint64_t &sq_bitmap, bool &cq_available))
- **MOCK_METHOD4** (waitForEpollEventsMasterMinion, void(int device, uint64_t &sq_bitmap, bool &cq_available, std::chrono::milliseconds timeout))
- **MOCK_METHOD2** (receiveResponseMasterMinion, bool(int device, std::vector< std::byte > &response))
- **MOCK_METHOD4** (sendCommandServiceProcessor, bool(int device, std::byte *command, size_t commandSize, CmdFlagSP flags))
- **MOCK_METHOD2** (setSqThresholdServiceProcessor, void(int device, uint32_t bytesNeeded))
- **MOCK_METHOD3** (waitForEpollEventsServiceProcessor, void(int device, bool &sq_available, bool &cq_available))
- **MOCK_METHOD4** (waitForEpollEventsServiceProcessor, void(int device, bool &sq_available, bool &cq_available, std::chrono::milliseconds timeout))
- **MOCK_METHOD2** (receiveResponseServiceProcessor, bool(int device, std::vector< std::byte > &response))
- **MOCK_CONST_METHOD1** (getSubmissionQueuesCount, int(int device))
- **MOCK_CONST_METHOD1** (getSubmissionQueueSizeMasterMinion, size_t(int device))
- **MOCK_CONST_METHOD1** (getSubmissionQueueSizeServiceProcessor, size_t(int device))
- **MOCK_CONST_METHOD1** (getDeviceStateMasterMinion, DeviceState(int device))
- **MOCK_CONST_METHOD1** (getDeviceStateServiceProcessor, DeviceState(int device))
- **MOCK_CONST_METHOD0** (getDmaAlignment, int())
- **MOCK_CONST_METHOD1** (getDramSize, uint64_t(int))
- **MOCK_CONST_METHOD1** (getDramBaseAddress, uint64_t(int))
- **MOCK_CONST_METHOD0** (getDevicesCount, int())

- **MOCK_CONST_METHOD2** (getDeviceAttribute, std::string(int, std::string))
- **MOCK_CONST_METHOD2** (clearDeviceAttributes, void(int, std::string))
- **MOCK_METHOD3** (allocDmaBuffer, void ∗(int, size_t sizeInBytes, bool))
- **MOCK_METHOD1** (freeDmaBuffer, void(void ∗))
- **MOCK_METHOD2** (getTraceBufferSizeMasterMinion, size_t(int, TraceBufferType))
- **MOCK_METHOD3** (getTraceBufferServiceProcessor, bool(int, TraceBufferType, std::vector< std::byte > &))
- **MOCK_METHOD1** (getDeviceConfig, DeviceConfig(int))
- **MOCK_METHOD1** (getActiveShiresNum, int(int))
- **MOCK_METHOD1** (getFrequencyMHz, uint32_t(int))
- **MOCK_METHOD2** (updateFirmwareImage, int(int, std::vector< unsigned char > &))
- **MOCK_CONST_METHOD0** (getFreeCmaMemory, size_t())
- **MOCK_CONST_METHOD1** (getDmaInfo, DmaInfo(int))
- **MOCK_METHOD3** (reinitDeviceInstance, void(int device, bool masterMinionOnly, std::chrono::milliseconds timeout))
- **MOCK_METHOD1** (hintInactivity, void(int))
- void **Delegate** ()

Additional Inherited Members

The documentation for this class was generated from the following file:

- DeviceLayerMock.h

5.5 dev::DmaInfo Struct Reference

This struct contains the limitations / optimal DMA parameters.

```
#include <IDeviceLayer.h>
```

Public Attributes

- uint64_t maxElementSize_

  maximum amount of memory that can be transfer per each DMA command entry
- uint64_t maxElementCount_

  max number of DMA entries per DMA command

5.5.1 Detailed Description

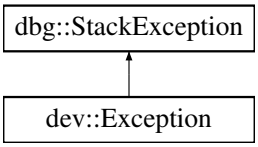
This struct contains the limitations / optimal DMA parameters.

The documentation for this struct was generated from the following file:

- IDeviceLayer.h

5.6 dev::Exception Class Reference

Inheritance diagram for dev::Exception:

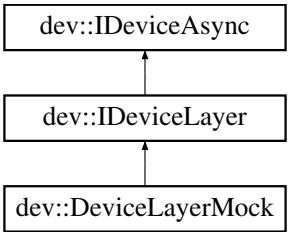


The documentation for this class was generated from the following file:

- IDeviceLayer.h

5.7 dev::IDeviceAsync Class Reference

Inheritance diagram for dev::IDeviceAsync:



Public Member Functions

- virtual bool sendCommandMasterMinion (int device, int sqIdx, std::byte ∗command, size_t commandSize, CmdFlagMM flags)=0

  Sends a command to the master minion. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsMasterMinion()`
- virtual void setSqThresholdMasterMinion (int device, int sqIdx, uint32_t bytesNeeded)=0

  Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeMasterMinion()`.
- virtual void waitForEpollEventsMasterMinion (int device, uint64_t &sq_bitmap, bool &cq_available, std↩ ::chrono::milliseconds timeout=std::chrono::seconds(10))=0

  Blocks the caller thread until submission queues or Completion Queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.
- virtual bool receiveResponseMasterMinion (int device, std::vector< std::byte > &response)=0

  Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsMasterMinion()`
- virtual bool sendCommandServiceProcessor (int device, std::byte ∗command, size_t commandSize, CmdFlagSP flags)=0

  Sends a command to the service processor. If the method returns false, the caller should try later when the queue has enough space indicated by availability from `waitForEpollEventsServiceProcessor()`
- virtual void setSqThresholdServiceProcessor (int device, uint32_t bytesNeeded)=0

Set the submission queue availability threshold. Submission queue epoll event will be generated only if space on submission queue is greater or equal to this threshold set. Default threshold value is one forth of size of submission queue buffer returned by `getSubmissionQueueSizeServiceProcessor()`.

- virtual void waitForEpollEventsServiceProcessor (int device, bool &sq_available, bool &cq_available, std↩::chrono::milliseconds timeout=std::chrono::seconds(10))=0

  Blocks the caller thread until submission queue or completion queue is available for use. The operation will be aborted if no event happen in given timeout. In such case, sq_bitmap and cq_available will be both 0 and false.
- virtual bool receiveResponseServiceProcessor (int device, std::vector< std::byte > &response)=0

  Receives a response from the device, a non-blocking interface. If the method returns false, the caller should try later when the queue has response available indicated by `waitForEpollEventsServiceProcessor()`
- virtual ~IDeviceAsync ()=default

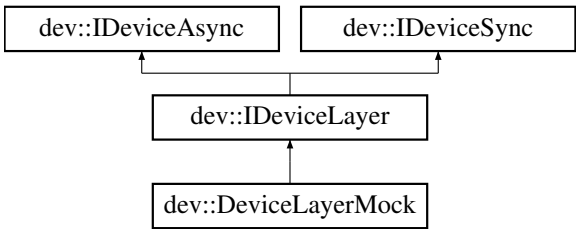
  Virtual Destructor to enable polymorphic release of the IDeviceAsync instances.

The documentation for this class was generated from the following file:

- IDeviceLayer.h

5.8 dev::IDeviceLayer Class Reference

Inheritance diagram for dev::IDeviceLayer:



Public Member Functions

- virtual ~IDeviceLayer ()=default

  Virtual Destructor to enable polymorphic release of the IDeviceApi instances.

Static Public Member Functions

- static std::unique_ptr< IDeviceLayer > createSysEmuDeviceLayer (const emu::SysEmuOptions &options, uint8_t numDevices=1)

  Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.
- static std::unique_ptr< IDeviceLayer > createSysEmuDeviceLayer (std::vector< emu::SysEmuOptions > options)

  Factory method to instantiate a IDeviceApi implementation, based on sysemu backend.
- static std::unique_ptr< IDeviceLayer > createPcieDeviceLayer (bool enableMasterMinion=true, bool enableServiceProcessor=false)

  Factory method to instantiate a IDeviceApi implementation, based on Pcie backend.

The documentation for this class was generated from the following file:

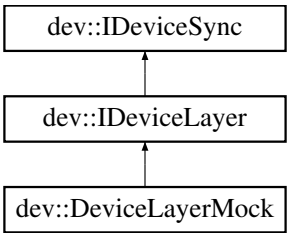
- IDeviceLayer.h

5.9 dev::IDeviceSync Class Reference

This should be a command Interface class which is shared between Ops and Management, although will be different.

```
#include <IDeviceLayer.h>
```

Inheritance diagram for dev::IDeviceSync:



Public Member Functions

- virtual DmaInfo getDmaInfo (int device) const =0

  Returns the DMA parameters.
- virtual int getDevicesCount () const =0

  Returns the number of esperanto devices connected to the host.
- virtual int getSubmissionQueuesCount (int device) const =0

  Returns the number of submission queues associated to given device.
- virtual DeviceState getDeviceStateMasterMinion (int device) const =0

  Gets device state associated to given Master Minion device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::NotResponding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.
- virtual DeviceState getDeviceStateServiceProcessor (int device) const =0

  Gets device state associated to given Service Processor device and returns enum DeviceState If the caller didn't send any command in current session and received DeviceState::PendingCommands, then caller may choose to reset the device by sending abort commands and discard any previous responses. If received DeviceState::NotResponding, then caller should try to recover by sending abort commands. Caller should be careful in polling on this method because it directly gets status from device and hence can be expensive in I/O transactions with device.
- virtual size_t getSubmissionQueueSizeMasterMinion (int device) const =0

  Returns the submission queue buffer size in bytes associated to given device all submission queues on Master Minion are equal in size.
- virtual size_t getSubmissionQueueSizeServiceProcessor (int device) const =0

  Returns the submission queue buffer size in bytes associated to given device.
- virtual size_t getTraceBufferSizeMasterMinion (int device, TraceBufferType traceType)=0

  Receives Master Minion's trace buffer sizes associated to given device.
- virtual bool getTraceBufferServiceProcessor (int device, TraceBufferType traceType, std::vector< std::byte > &traceBuf)=0

  Receives Service Processor's trace buffer associated to given device in traceBuf.
- virtual int updateFirmwareImage (int device, std::vector< unsigned char > &fwImage)=0

  Writes firmware image on DRAM of given device.
- virtual int getDmaAlignment () const =0

  Returns the DMA alignment requirement.
- virtual uint64_t getDramSize (int device) const =0

  Returns the DRAM available size (in bytes)

- virtual uint64_t getDramBaseAddress (int device) const =0

  Returns the DRAM base address.
- virtual ~IDeviceSync ()=default

  Virtual Destructor to enable polymorphic release of the IDeviceOpsSync instances.
- virtual void * allocDmaBuffer (int device, size_t sizeInBytes, bool writeable)=0

  Allocates a consecutive chunk of memory, able to do DMA. This chunk of memory must be deallocated properly using the freeDmaBuffer function.
- virtual void freeDmaBuffer (void *dmaBuffer)=0

  Deallocates a previously allocated dmaBuffer.
- virtual DeviceConfig getDeviceConfig (int device)=0

  returns a DeviceConfig struct containing device configured details
- virtual int getActiveShiresNum (int device)=0

  returns a number of active shires (number of bits set to 1 in shire mask)
- virtual uint32_t getFrequencyMHz (int device)=0

  returns device frequency
- virtual size_t getFreeCmaMemory () const =0

  returns the available CMA memory in the system. That memory is allocated using allocDmaBuffer
- virtual std::string getDeviceAttribute (int device, std::string relAttrPath) const =0

  Get device statistics by reading the attribute files.
- virtual void clearDeviceAttributes (int device, std::string relGroupPath) const =0

  Clear device statistics in the attribute files.
- virtual void reinitDeviceInstance (int device, bool masterMinionOnly, std::chrono::milliseconds timeout)

  Reinitialize the device master minion or both master minion and service processor NOTE: During master minion reinitialization other master minion APIs cannot be used. During both devices' reinitialization no other API should be used.
- virtual void hintInactivity (int device)

  Hints the underlying device that we don't have anything to process, so it could potentially go idle / low power consumption.

5.9.1 Detailed Description

This should be a command Interface class which is shared between Ops and Management, although will be different.

The documentation for this class was generated from the following file:

- IDeviceLayer.h

Index

allocDmaBuffer
    Device API, 11

clearDeviceAttributes
    Device API, 11
createPcieDeviceLayer
    Device API, 11
createSysEmuDeviceLayer
    Device API, 12

dev::CmdFlagMM, 25
dev::CmdFlagSP, 25
dev::DeviceConfig, 26
dev::DeviceLayerMock, 27
dev::DmaInfo, 28
dev::Exception, 29
dev::IDeviceAsync, 29
dev::IDeviceLayer, 30
dev::IDeviceSync, 31
Device API, 7
    allocDmaBuffer, 11
    clearDeviceAttributes, 11
    createPcieDeviceLayer, 11
    createSysEmuDeviceLayer, 12
    freeDmaBuffer, 13
    getActiveShiresNum, 13
    getDeviceAttribute, 13
    getDeviceConfig, 14
    getDevicesCount, 14
    getDeviceStateMasterMinion, 14
    getDeviceStateServiceProcessor, 14
    getDmaAlignment, 15
    getDmaInfo, 15
    getDramBaseAddress, 15
    getDramSize, 17
    getFreeCmaMemory, 17
    getFrequencyMHz, 17
    getSubmissionQueuesCount, 18
    getSubmissionQueueSizeMasterMinion, 18
    getSubmissionQueueSizeServiceProcessor, 18
    getTraceBufferServiceProcessor, 18
    getTraceBufferSizeMasterMinion, 19
    hintInactivity, 19
    receiveResponseMasterMinion, 19
    receiveResponseServiceProcessor, 20
    reinitDeviceInstance, 20
    sendCommandMasterMinion, 21
    sendCommandServiceProcessor, 21
    setSqThresholdMasterMinion, 22
    setSqThresholdServiceProcessor, 22
    updateFirmwareImage, 22
    waitForEpollEventsMasterMinion, 23
    waitForEpollEventsServiceProcessor, 23

freeDmaBuffer
    Device API, 13

getActiveShiresNum
    Device API, 13
getDeviceAttribute
    Device API, 13
getDeviceConfig
    Device API, 14
getDevicesCount
    Device API, 14
getDeviceStateMasterMinion
    Device API, 14
getDeviceStateServiceProcessor
    Device API, 14
getDmaAlignment
    Device API, 15
getDmaInfo
    Device API, 15
getDramBaseAddress
    Device API, 15
getDramSize
    Device API, 17
getFreeCmaMemory
    Device API, 17
getFrequencyMHz
    Device API, 17
getSubmissionQueuesCount
    Device API, 18
getSubmissionQueueSizeMasterMinion
    Device API, 18
getSubmissionQueueSizeServiceProcessor
    Device API, 18
getTraceBufferServiceProcessor
    Device API, 18
getTraceBufferSizeMasterMinion
    Device API, 19

hintInactivity
    Device API, 19

receiveResponseMasterMinion
    Device API, 19
receiveResponseServiceProcessor
    Device API, 20
reinitDeviceInstance

Device API, 20

sendCommandMasterMinion
Device API, 21
sendCommandServiceProcessor
Device API, 21
setSqThresholdMasterMinion
Device API, 22
setSqThresholdServiceProcessor
Device API, 22

updateFirmwareImage
Device API, 22

waitForEpollEventsMasterMinion
Device API, 23
waitForEpollEventsServiceProcessor
Device API, 23